

UNIVERSITATEA „ȘTEFAN CEL MARE”, SUCEAVA

FACULTATEA DE INGINERIE ELECTRICĂ ȘI ȘTIINȚA CALCULATOARELOR

SPECIALIZAREA CALCULATOARE

AN 2, GRUPA 3121 B

PROIECT: PROGRAMARE ORIENTATA PE OBIECTE

Sistem Inteligent de Gestiune și Vânzări pentru Multiplex Cinema

Tehnologie de implementare: C++ / Biblioteca grafică de terminal ncurses

Autor: Solcanu Vlăduț Constantin

Profesor coordonator: Asist. Univ. Dr. Ing. Prodan Remus

Anul universitar: 2026

REZUMATUL PROIECTULUI (ABSTRACT)

Prezenta lucrare detaliază dezvoltarea unui sistem software integrat, destinat managementului operațional și financiar al unui cinematograful multiplex. Dezvoltată integral în limbajul **C++**, aplicația inovează prin utilizarea bibliotecii ncurses pentru a crea o **Interfață Grafică Text (TUI)** avansată. Aceasta transformă consola clasică într-un mediu interactiv fluid, controlabil complet prin evenimente de mouse (click și scroll), eliminând necesitatea introducerii comenzilor textuale.

Din punct de vedere arhitectural, sistemul utilizează o **Arhitectură pe Straturi (N-Tier)** care decuplează strict interfața grafică de logica de business și de persistența datelor. Operând pe o paradigmă **Data-Driven**, aplicația își extrage regulile (cataloge, săli, promoții) din fișiere text locale (flat-files), garantând o funcționare rapidă, de tip *offline*, fără a depinde de servere externe sau baze de date masive.

Platforma este structurată în două module principale:

- **Modulul Client:** Oferă o experiență intuitivă pentru selecția vizuală a scaunelor pe o hartă matricială interactivă, validând disponibilitatea instantaneu. Totodată, gestionează un coș de cumpărături ce aplică automat prețuri dinamice și oferte (ex. *Happy Hour*).
- **Modulul Administrator:** Securizat prin parolă, acest modul funcționează ca un panou de audit. Folosind algoritmi de agregare bazați pe structuri `std::map` (cu chei de sortare cronologică), sistemul comasează tranzacțiile financiare brute în rapoarte de vânzări clare și concise.

În concluzie, proiectul demonstrează eficiența conceptelor de Programare Orientată pe Obiecte (POO) în crearea unui software comercial extrem de rapid, scalabil și cu o amprentă hardware minimă.

CUPRINS:

REZUMATUL PROIECTULUI (ABSTRACT)	1
CAPITOLUL I: Introducere	2
CAPITOLUL II: Analiza Cerințelor și Detalii de Implementare Funcțională	5
CAPITOLUL IV: Arhitectura Sistemului și Descrierea Componentelor	11
CAPITOLUL V: Modelarea Sistemului (Diagrame UML)	14
CAPITOLUL VI: Implementarea Sistemului.....	17
CAPITOLUL VII: Concluzii și Direcții Viitoare de Dezvoltare	20
BIBLIOGRAFIE	25

CAPITOLUL I: Introducere

Prezenta lucrare detaliază analiza, proiectarea, implementarea și testarea unui sistem software complex, dezvoltat integral în limbajul C++, destinat managementului operațional și financiar al unui cinematograful de tip multiplex. Proiectul explorează integrarea conceptelor avansate de Programare Orientată pe Obiecte (POO) cu dezvoltarea de interfețe grafice de terminal (TUI), rezultând într-o platformă rapidă, stabilă și independentă de limitările aplicațiilor comerciale tradiționale.

1.1 Titlul proiectului

Sistem Inteligent de Gestiune și Vânzări pentru Multiplex Cinema

1.2 Scopul aplicației

Scopul fundamental al acestui proiect este de a oferi o soluție software unificată (All-in-One) care să automatizeze fluxurile principale dintr-un cinematograful: vânzarea de bilete, gestionarea hărții fizice a locurilor, comercializarea produselor complementare (Snack Bar) și generarea rapoartelor financiare de audit.

Spre deosebire de aplicațiile clasice de consolă care se bazează pe introducerea secvențială a textului de la tastatură (CLI), scopul tehnic al acestui sistem este de a aduce ergonomia și viteza unei interfețe grafice moderne (GUI) direct în terminal, utilizând biblioteca ncurses pentru a permite navigarea complet asincronă, exclusiv prin evenimente de mouse (click și scroll). Mai mult, aplicația își propune să elimine logica hardcodată, funcționând pe baza unei arhitecturi *Data-Driven*, în care toate regulile de business sunt dictate de fișiere de date externe.

1.3 Contextul problemei

Industria cinematografică operează cu volume mari de tranzacții concentrate în ferestre scurte de timp (de exemplu, cu 15 minute înaintea începerii unui blockbuster). În acest context, sistemele de gestiune (POS - Point of Sale) trebuie să fie extrem de rapide și fiabile.

Problemele sistemelor actuale pe care acest proiect încearcă să le rezolve sunt:

- **Consumul uriaș de resurse și latență:** Sistemele web-based sau cele dezvoltate în framework-uri greoaie (Java, .NET, Electron) necesită hardware scump și conexiune permanentă la internet. În cazul unei căderi de rețea, vânzările sunt complet blocate.
- **Eroarea umană în aplicarea promoțiilor:** Casierii trebuie adesea să aplice manual reduceri (ex. bilete mai ieftine vineri, sau meniuri Happy Hour), ceea ce duce la erori de încasare și pierderi financiare.
- **Lipsa de flexibilitate a aplicațiilor legacy:** Programele mai vechi de consolă sunt robuste, dar extrem de lente în operare, deoarece forțează angajatul să tasteze coduri de filme sau numere de scaune.

1.4 Obiectivele proiectului

Pentru a răspunde problemelor identificate în contextul actual, proiectul a fost divizat într-un set de obiective tehnice și operaționale clare:

1. **Dezvoltarea unei Interfețe TUI Interactive:** Implementarea unui mediu vizual fluid în

terminal, controlabil 100% prin mouse, utilizând primitive grafice, perechi de culori și sisteme de ferestre glisante (paginare prin scroll).

2. **Arhitectură Decuplată pe Straturi (N-Tier):** Separarea strictă a logicii de vizualizare (Ecrane.h) de logica datelor (BazaDate.h) și de controlerul principal, respectând principiile ingineriei software (SOLID).
3. **Implementarea unui Motor de Prețuri Dinamice:** Crearea unui algoritm matematic capabil să evalueze timpul real (ziua și ora sistemului) și să ajusteze automat prețurile din coșul de cumpărături pe baza unor reguli definite în fișiere externe, eliminând intervenția umană.
4. **Sistem de Persistență Locală (Flat-file Database):** Proiectarea unui mecanism sigur și rapid de citire/scriere pe disc folosind fișiere text delimitate (|), asigurând capacitatea de operare *Offline* a cinematografului 24/7.
5. **Audit Inteligent:** Dezvoltarea unui algoritm de agregare și sortare cronologică forțată (utilizând `std::map` cu chei injectate) care să comaseze tranzacțiile financiare pentru rapoartele de management.

1.5 Publicul țintă (Grupuri de utilizatori)

Aplicația a fost concepută având în minte trei categorii distincte de utilizatori, fiecare beneficiind de funcționalități specifice nivelului lor de acces:

- **Clienții cinematografului (Modul Kiosk/Self-Service):** Interfața este suficient de intuitivă și protejată împotriva erorilor încât poate fi rulată pe un ecran tactil în holul cinematografului. Clienții pot naviga prin programul filmelor, pot vizualiza interactiv harta sălii pentru a-și alege singuri locurile (evitând scaunele ocupate, marcate cu roșu) și își pot asambla coșul de cumpărături cu bilete și produse de la bar, beneficiind în mod transparent de ofertele active.
- **Angajații cinematografului (Casieri / Operatori POS):** Pentru personalul de la casele de marcat, viteza este vitală. Interacțiunea asincronă bazată pe mouse le permite casierilor să selecteze locurile dorite de clienți în fracțiuni de secundă. Sistemul automatizat le preia povara calculării reducerilor, reducând stresul operațional și timpul de așteptare la coadă.
- **Administratorii și Managerii (Back-Office):** Reprezintă personalul de decizie. Prin intermediul unui panou ascuns și securizat cu parolă, managerii pot accesa modulul de raportare. Aceștia nu sunt copleșiți de mii de linii de tranzacții brute, ci primesc rapoarte agregate și sumarizate inteligent, esențiale pentru contabilitate, reconcilierea încasărilor și analiza performanței cinematografului.

CAPITOLUL II: Analiza Cerințelor și Detalii de Implementare Funcțională

1. Înregistrare și autentificare utilizatori

Implementarea securității și a autentificării a fost dezvoltată prioritar pentru modulul administrativ (Back-Office).

- **Logică de control:** Accesul este controlat prin starea ADMIN_LOGIN din bucla principală a aplicației.
- **Mecanism vizual:** La tastarea parolei, sistemul interceptează caracterele prin funcția getch() și generează vizual o mască de securitate sub formă de asteriscuri (string masca(parolaIntrodusa.length(), '*');), prevenind expunerea pe ecran.
- **Validare:** Se face apelând metoda BazaDate::verificaParola(), care citește și compară șirul de caractere cu valoarea stocată securizat în fișierul parola.txt. Trecerea către panoul de control (ADMIN_MENIU) este permisă strict la potrivirea exactă a credențialelor.

```
void afiseazaLoginAdmin() {
    mvprintw(LINES / 2 - 2, COLS / 2 - 15, "Introduceti parola de acces:");
    string parolaIntrodusa = "";
    int ch;

    // Citire caracter cu caracter pentru securitate
    while ((ch = getch()) != '\n') {
        if (ch == KEY_BACKSPACE || ch == 127 || ch == '\b') {
            if (!parolaIntrodusa.empty()) {
                parolaIntrodusa.pop_back();
            }
        } else {
            parolaIntrodusa += (char)ch;
        }
    }

    // Randarea măștii de securitate
    string masca(parolaIntrodusa.length(), '*');
    mvprintw(LINES / 2, COLS / 2 - 15, " "); // clear line
    mvprintw(LINES / 2, COLS / 2 - 15, "%s", masca.c_str());
}
```

Figura 2.1 Mascarea parolei administratorului

2. Vizualizare filme disponibile

Rularea catalogului de filme este realizată prin extragerea datelor de către metoda `BazaDate::incarcaFilme()`, care folosește `std::ifstream` pentru a parsă fișierul `filme.txt`.

- **Stocare în memorie:** Entitățile extrase sunt stocate sub forma unui vector dinamic `vector<Film>`.
- **Randare grafică:** Stratul de prezentare iterează prin acest vector și desenează informațiile pe ecran folosind macrourile `ncurses` (`mvprintw`).
- **Stilizare:** Sistemul aplică dinamic perechi de culori (`COLOR_PAIR`) și inserează logic etichete grafice pre-formate, cum ar fi `[PREMIERA]`, în funcție de flag-urile booleene setate în atributele obiectelor `Film`.

3. Căutare și filtrare filme

Funcția de filtrare este implementată temporal (*time-aware filtering*).

- **Procesare la runtime:** În cadrul buclei principale `while(ruleaza)`, programul utilizează biblioteca standard `<ctime>` pentru a interoga timpul curent al sistemului de operare.
- **Algoritm de curățare:** Programul iterează prin vectorul proiecțiilor (`programAzi`) și calculează diferența în minute dintre ora curentă și ora de difuzare. Funcția `programAzi.erase(it)` elimină automat din memorie și de pe ecran orice proiecție care a depășit timpul de toleranță (30 de minute peste ora de început).
- **Optimizare interfață:** Navigarea prin lista rămasă se face printr-un algoritm de paginare controlat de un offset, care glisează fereastra de vizualizare pe baza evenimentelor de scroll ale mouse-ului (`BUTTON4_PRESSED` / `BUTTON5_PRESSED`).

4. Vizualizare program cinema

Programul este generat printr-o operațiune de tip "JOIN" implementată direct în memorie în cadrul clasei `Proiectie`.

- **Asamblare date:** Metoda `BazaDate::incarcaProgram()` citește fișierul `program.txt` și assemblează instanțe de tip `Proiectie`, corelând un obiect `Film` și un obiect `Sala` pe baza ID-urilor unice și a unui șir de caractere reprezentând ora.
- **Avertizări dinamice:** Pentru a atrage atenția utilizatorului, stratul de vizualizare modifică automat culoarea butoanelor grafice. Dacă un film urmează să înceapă în mai puțin de 30 de minute, zona interactivă a butonului va fi randată cu o avertizare vizuală roșie (`COLOR_PAIR(4)`), trecând starea aplicației în `AVERTISMENT_INTARZIERE` la

declanșarea evenimentului de click.

5. Rezervare locuri (Seat Mapping)

Desenarea sălii de cinema este o reprezentare vizuală a unei matrici virtuale. Fiecare instanță a clasei Sala conține dimensiunile acestei matrici (rânduri și coloane). În starea SELECTIE_LOCURI, algoritmul calculează lățimea totală a terminalului (COLS) și centrează dinamic scaunele pe ecran.

Logica de disponibilitate verifică intersecțiile cu datele din fișierul rezervari_locuri.txt. Matricea este transpusă vizual utilizând culori și simboluri specifice:

- **[01] (Verde/Galben)** – Loc liber; poate fi selectat și adăugat în structura curentă locuriSelectateCurent.
- **[XX] (Roșu)** – Loc salvat deja în baza de date; interceptia evenimentelor de mouse este dezactivată complet pentru aceste coordonate.
- **[SS] (Alb intens)** – Loc rezervat temporar în sesiunea curentă, aflat în așteptarea confirmării finale de plată.

6. Achiziție bilete (și produse complementare)

Acest proces are loc în starea COS, fiind controlat la nivel de backend de structura vector<ProdusCos> cos.

- **Calcularea dinamică a prețurilor:** Toate obiectele din coș trec printr-un motor de prețuri apelat la runtime (obținePretBiletDinamic / obținePretBarDinamic). Acesta extrage regulile active din fișierul promotii.txt și aplică polimorfic fie o reducere procentuală ($\text{preț} * (1 - \text{reducere}/100)$), fie o forțare de preț fix.
- **Persistența tranzacției:** La acționarea butonului „Finalizează Comanda”, aplicația apelează BazaDate::finalizeazaComanda(). Aceasta instanțiază fluxuri std::ofstream în modul append (ios::app) pentru a serializa conținutul vectorului în fișierele corespunzătoare (vanzari_bilete.txt, vanzari_bar.txt), eliberând ulterior memoria locală.

7. Gestionare profil utilizator (Sesiunea Curentă)

În arhitectura actuală, profilul clientului ia forma gestionării stricte a stării sesiunii (*Session Management*).

- **Starea sesiunii:** Pe parcursul navigării, clientul acumulează date în vectorul temporar cos și coordonate în structura vector<pair<int, int>> locuriSelectateCurent.

- **Mecanismul de Rollback (Anulare):** Sistemul îi permite utilizatorului manipularea activă a propriei sesiuni. La apăsarea butonului de ștergere a unui bilet din coș (identificat prin coordonata grafică [X]), metoda `programAzi[index].elibereazaLoc(rand, loc)` este apelată imediat. Aceasta eliberează matricea de disponibilitate din memoria RAM și reface starea inițială a scaunului pe harta interactivă.

8. Administrare filme și programări (Gestiune Back-Office)

Această cerință se materializează în modulele analitice `ADMIN_BILETE` și `ADMIN_BAR`.

- **Optimizarea performanței:** Soluția tehnică pentru agregarea rapidă a datelor financiare evită citirea repetitivă a discului fizic, executând o singură trecere peste fișiere și stocând datele optimizat în structuri de tip `std::map`.
- **Algoritmul de injecție cronologică:** Deoarece structura de bază de tip `map` sortează datele strict alfabetic în C++, implementarea a impus crearea unei chei de sortare injectate cronologic: `string cheieSortare = YYYYMMDD + "|" + Ora + "|" + Produs;`
- **Generarea rapoartelor:** La iterarea hărții de date, aplicația reușește să aglomereze toate tranzacțiile identice din același minut și să le afișeze administratorului sub formă de rapoarte comasate (afișând cantități cumulate Buc: N și prețuri totale sumate), eliminând necesitatea calculelor manuale la nivelul bazei de date flat-file.

Capitolul III: Studiul Solutiei

1. Analiza Aplicatiilor Similare

În peisajul actual al industriei de entertainment, sistemele de gestiune cinematografică (*Cinema Management Systems* – CMS) se împart, în general, în două mari categorii tehnologice: aplicații desktop masive (dezvoltate în Java, C# WinForms sau WPF) și platforme Web/Cloud-based (dezvoltate utilizând framework-uri precum React, Angular și backend-uri Node.js sau Spring Boot).

Deși aceste soluții comerciale (precum sistemele Vista Cinema sau software-urile de tip ERP personalizat) oferă o multitudine de funcționalități, ele prezintă o serie de limitări și dezavantaje tehnice în anumite medii de operare:

- **Consum ridicat de resurse (Overhead hardware):** Aplicațiile bazate pe tehnologii Web (Electron, browsere integrate) sau mașini virtuale (JVM, .NET CLR) necesită o cantitate semnificativă de memorie RAM și putere de procesare. Acest lucru obligă cinematografele să investească în terminale POS (*Point of Sale*) costisitoare pentru casele de bilete.

- **Dependența de rețea (Network Latency):** Soluțiile Cloud-based devin nefuncționale sau prezintă latențe severe în momentul în care conexiunea la internet a locației fizice devine instabilă, blocând complet capacitatea de a vinde bilete la fața locului.
- **Complexitatea instalării și a mentenanței:** Sistemele tradiționale necesită instalarea și configurarea prealabilă a unor servere de baze de date masive (Microsoft SQL Server, Oracle, MySQL), a unor runtime-uri specifice și a unor licențe costisitoare.

2. Avantajele Soluției Propuse

Sistemul dezvoltat în acest proiect abordează problema dintr-o perspectivă diferită, combinând eficiența și viteza programării la nivel de sistem cu o interfață utilizator ergonomică. Soluția propusă iese în evidență prin următoarele avantaje tehnice și operaționale majore:

- **Performanță extremă și amprentă minimă de memorie (Lightweight):** Rulând nativ ca o aplicație executabilă de terminal (C++ compilat), sistemul consumă o fracțiune din memoria RAM necesară unei aplicații web. Acest lucru permite rularea impecabilă a programului pe echipamente POS vechi (*legacy hardware*) sau chiar pe micro-calculatoare (ex. Raspberry Pi).
- **Interfață TUI (*Text-based User Interface*) bazată pe evenimente:** Spre deosebire de aplicațiile clasice de consolă care forțează utilizatorul să tasteze comenzi secvențiale, soluția de față utilizează un mecanism asincron de interceptare a mouse-ului. Angajatul poate da click pe locurile din sală, poate face scroll prin liste și poate interacționa cu butoane grafice la fel ca într-un GUI modern, crescând masiv viteza de procesare a clienților.
- **Arhitectură Data-Driven și Decuplată:** Sistemul nu necesită recompilare pentru a modifica prețurile, a adăuga filme sau a schimba structura ofertelor. Toate politicile de business sunt citite la runtime din fișiere, oferind o flexibilitate similară cu a unui sistem CMS profesional.
- **Reziliență și operare Offline:** Neavând dependențe de un server Cloud extern sau de un motor RDBMS greoi, aplicația citește și scrie stările tranzacționale direct pe discul local în format flat-file. Această abordare garantează funcționarea neîntreruptă a casei de bilete chiar și în scenariul unei căderi totale a rețelei internet.

3. Tehnologiile Alese și Justificarea Lor

Procesul de selecție a stivei tehnologice (*Tech Stack*) a fost ghidat de cerințele stricte de

performanță, portabilitate și de necesitatea de a implementa riguros concepte de programare orientată pe obiecte.

3.1 Limbajul de programare C++ (Standardul C++11/14):

- **Justificare:** C++ a fost ales ca limbaj principal de dezvoltare datorită controlului determinist asupra memoriei și a vitezei de execuție de neegalat.
- **Utilizare:** S-a utilizat intens *Standard Template Library* (STL) pentru gestiunea dinamică a datelor. Structurile de tip `std::vector` au permis manipularea eficientă a cataloagelor variabile de filme, iar `std::map` a facilitat indexarea avansată și sortarea cheie-valoare necesară pentru algoritmi de agregare cronologică a istoricului de vânzări. Polimorfismul și încapsularea au asigurat o separare clară a modelelor de domeniu (Film, Sala, Proiecție).

3.2. Biblioteca grafică NCURSES (New Curses):

- **Justificare:** Stream-urile standard (`std::cin`, `std::cout`) blochează execuția programului (*I/O blocking*) și randează textul doar liniar, de sus în jos. Biblioteca `ncurses` a fost selectată deoarece oferă un API robust pentru controlul ecranului la nivel de caracter și celulă (coordonate x, y).
- **Funcționalități implementate:** Această tehnologie a permis:
 - Desenarea matricelor bidimensionale pentru hărțile sălilor de cinema.
 - Utilizarea perechilor de culori (`init_pair`) pentru a oferi un feedback vizual imediat (ex. roșu pentru avertismente, verde pentru acțiuni valide).
 - Hooking-ul evenimentelor hardware prin masca `mousemask`, facilitând interceptarea click-urilor și a scroll-ului, transformând o simplă consolă într-un panou interactiv complex.

```
void proceseazaEvenimenteInput() {  
    // Activare mască pentru interceptarea absolută a mouse-ului  
    mousemask(ALL_MOUSE_EVENTS | REPORT_MOUSE_POSITION, NULL);  
  
    int c = getch();  
    if (c == KEY_MOUSE) {  
        MEVENT event;  
        if (getmouse(&event) == OK) {  
            // Verificare Scroll  
            if (event.bstate & BUTTON4_PRESSED) {  
                if (offsetCurent > 0) offsetCurent--;  
            } else if (event.bstate & BUTTON5_PRESSED) {  
                offsetCurent++;  
            }  
            // Verificare Click Stânga  
            else if (event.bstate & BUTTON1_CLICKED) {  
                verificaClickButoane(event.x, event.y);  
            }  
        }  
    }  
}
```

Figura 3.3.2. Inițializarea și interceptarea Mouse-ului

3.3. Sistem de persistență Flat-File (Fișiere text delimitate):

- **Justificare:** Pentru a păstra aplicația portabilă, modulară și fără dependențe de instalare, s-a evitat utilizarea unui motor SQL tradițional în această etapă a dezvoltării. Fișierele de tip .txt au fost folosite ca mediu de persistență (*Flat-file Database*).
- **Utilizare:** S-a implementat o logică de parsare a șirurilor de caractere folosind `std::stringstream` și funcția `getline()` cu delimitatorul pipe (`|`). Această abordare permite administratorilor sistemului să modifice rapid configurațiile promoțiilor sau să auditeze log-urile de vânzări folosind orice editor de text standard, asigurând în același timp o latență de acces la I/O extrem de redusă.

3.4. Biblioteca Standard <ctime>:

- **Justificare:** Operarea unui cinema este strict dependentă de factorul temporal.
- **Utilizare:** Funcțiile POSIX din <ctime> au fost integrate pentru a genera structuri de tip `tm*`. Acest lucru a permis implementarea unor filtre temporale dinamice (*time-aware filtering*), cum ar fi calcularea diferenței absolute în minute pentru avertismentele de întârziere la proiecții sau aplicarea automată a ofertelor de tip "Happy Hour" în funcție de ziua săptămânii extrasă din sistemul de operare (`tm_wday`).

CAPITOLUL IV: Arhitectura Sistemului și Descrierea Componentelor

Pentru a asigura o funcționare stabilă, o mentenanță facilă și o capacitate ridicată de extindere, aplicația a fost proiectată renunțând la abordarea monolitului tradițional în favoarea unei **Arhitecturi pe Straturi** (*Layered Architecture / N-Tier Architecture*). Această decizie inginerească garantează principiul decuplării (*Separation of Concerns*), în care fiecare componentă software are o responsabilitate unică și bine definită.

Sistemul este divizat în patru componente (straturi) majore, care comunică între ele secvențial, de sus în jos.

4.1. Stratul de Prezentare (Presentation Layer / View):

- **Modulul aferent:** `Ecrane.h`
- **Descriere:** Acest strat este responsabil exclusiv cu generarea și manipularea interfeței

grafice în consolă (TUI) pe care o vede utilizatorul. Este o componentă de tip *"dumb view"* (vedere pasivă), ceea ce înseamnă că nu conține logică de business sau calcule financiare, ci doar randează datele pe care le primește de la nivelurile inferioare.

- **Tehnologie:** Utilizează direct primitivele bibliotecii ncurses.
- **Funcționalități principale:**
 - Gestionează culorile interfeței prin definirea și aplicarea perechilor de culori (COLOR_PAIR).
 - Desenează componentele grafice de bază: butoane interactive, ferestre de dialog, grile pentru meniul de bar și matricea bidimensională reprezentând harta locurilor din sală.
 - Maschează elementele de securitate (parolele).
 - Gestionează paginarea vizuală (randarea unui subset de date pe baza unui parametru de tip offset), adaptând ieșirea la dimensiunea variabilă a terminalului (COLS și LINES).

4.2. Stratul de Control și Rutare (Application Controller):

- **Modulul aferent:** main.cpp (Controller)
- **Descriere:** Acest strat acționează ca un "creier" operațional sau un dispecerat al aplicației. Inspirat din design pattern-ul *Game Loop* specific dezvoltării de jocuri video, acest modul menține ciclul de viață al programului.
- **Componente și mecanisme cheie:**
 - **Mașina de Stări Finite (FSM - Finite State Machine):** Controlează fluxul de navigare folosind enumerarea StareAplicatie (ex: MENIU_PRINCIPAL, BILETE, SELECTIE_LOCURI, COS). Fiecare iterație a buclei evaluează starea curentă și decide ce ecran trebuie apelat din stratul de prezentare.
 - **Event Handler (Gestionarea Evenimentelor):** Interceptează întreruperile hardware. Analizează coordonatele (x, y) ale click-urilor de mouse folosind structura MEVENT furnizată de ncurses și preia input-ul de la tastatură (prin getch()).
 - **Sesiunea Curentă:** Acționează ca un cache temporar. Păstrează în memoria volatilă (RAM) coșul de cumpărături al utilizatorului curent (vector<ProdusCos> cos) și locurile selectate (vector<pair<int, int>> locuriSelectateCurent), până la momentul în care tranzacția este confirmată (*Commit*) sau anulată (*Rollback*).

4.3. Stratul de Domeniu (Business Logic / Domain Model):

- **Modulele aferente:** Film.h, Sala.h, Proiectie.h, Gestiune.h
- **Descriere:** Aici rezidă "inteligența" aplicației. Acest strat conține clasele C++ care modelează conceptele din lumea reală a cinematografiei, încapsulând atributele și comportamentele acestora conform principiilor Programării Orientate pe Obiecte (POO).
- **Entități modelate și componente:**
 - **Entitatea Film:** Stochează proprietățile de bază ale peliculei (titlu, durată, gen, preț standard, flag de premieră).
 - **Entitatea Sala:** Gestionează topologia fizică a locației. Implementează logica spațială, definind numărul de rânduri, coloane, culoarele de trecere și clasificarea sălii (Standard, IMAX, VIP).
 - **Entitatea Proiectie:** Este entitatea centrală care corelează un obiect Film cu un obiect Sala la o anumită instanță de timp (oră). Această clasă gestionează dinamic o structură de date matriceală de tip boolean, responsabilă de marcarea stării fiecărui scaun (rezervat/liber) și expune metode de manipulare a stării (rezervaLoc, elibereazaLoc).
 - **Motorul de Promoții (Gestiune.h):** O componentă crucială a logicii de business. Calculează polimorfic prețurile finale (dinamice) aplicând reduceri procentuale sau sume fixe în funcție de evaluarea variabilelor de mediu (ziua săptămânii, ora curentă).

```
class Proiectie {
private:
    Film film;
    Sala sala;
    string oraFormatata;

    // Matrice dinamică bidimensională pentru gestionarea scaunelor
    vector<vector<bool>> locuriOcupate;

public:
    Proiectie(Film f, Sala s, string ora) : film(f), sala(s), oraFormatata(ora) {
        // Alocare memorie pentru matricea sălii
        locuriOcupate.resize(sala.getRanduri(),
                             vector<bool>(sala.getLocuriPeRand(), false));
    }

    bool esteLocLiber(int rand, int loc) const {
        return !locuriOcupate[rand][loc];
    }

    void rezervaLoc(int rand, int loc) {
        locuriOcupate[rand][loc] = true;
    }
};
```

Figura 4.3. Clasa Proiectie (Modelul matricial)

4.4. Stratul de Acces la Date (Data Access Layer / Persistence):

- **Modulul aferent:** BazaDate.h
- **Descriere:** Pentru a respecta principiul decuplării, niciunul dintre straturile superioare nu este conștient de modul în care datele sunt stocate fizic. Acest strat izolează complet operațiunile de intrare/ieșire (I/O) pe disc.
- **Operațiuni principale:**
 - **Deserializare (Citire):** Utilizează fluxuri de tip ifstream pentru a citi fișierele text linie cu linie. Parsează șirurile de caractere folosind std::stringstream și funcția getline() cu delimitatorul specific (pipe |), assemblează obiectele de domeniu (ex. Film, Proiectie) și le returnează către controler sub formă de vectori dinamic (std::vector).
 - **Serializare (Scriere):** La finalizarea unei comenzi, acest modul preia structurile de date din memoria volatilă (coșul de cumpărături), instanțiază fluxuri ofstream în modul append (ios::app) și transformă datele înapoi în șiruri de text delimitate, garantând persistența operațiunilor de rezervare și a tranzacțiilor financiare chiar și după închiderea aplicației.
 - **Agregare Analitică:** Conține algoritmi complecși bazați pe structura std::map pentru a extrage istoricul brut, a injecta chei de sortare cronologică și a comasa datele financiare înainte de a le trimite spre vizualizare către modulul de *Back-Office* al administratorului.

CAPITOLUL V: Modelarea Sistemului (Diagrame UML)

Pentru a oferi o imagine de ansamblu clară asupra arhitecturii, comportamentului și interacțiunilor din cadrul aplicației, sistemul a fost modelat vizual utilizând standardul **UML** (*Unified Modeling Language*). Această etapă de proiectare facilitează înțelegerea fluxurilor logice înainte de scrierea efectivă a codului sursă.

5.1. Diagrama Cazurilor de Utilizare (Use Case Diagram):

Diagrama cazurilor de utilizare definește granițele sistemului și ilustrează interacțiunile posibile dintre actorii externi și funcționalitățile aplicației. În cadrul sistemului nostru, au fost identificați doi actori principali cu permisiuni și fluxuri de lucru complet separate:

- **Actorul „Client” (Utilizator standard):** Reprezintă vizitatorul cinematografului. Acest actor interacționează cu sistemul pentru a realiza următoarele acțiuni (*Use Cases*):
 - Vizualizare filme disponibile;
 - Afișare program cinema;
 - Rezervare locuri pe harta interactivă;
 - Adăugare produse bar în coș;
 - Anulare rezervare (prin eliminarea produselor din coșul de cumpărături înainte de finalizare).

- **Actorul „Administrator” (Angajat/Manager):** Reprezintă personalul autorizat. Acțiunile sale includ:
 - Login (Autentificare securizată);
 - Vizualizare rapoarte financiare (Bilete și Bar);
 - *Dezvoltare ulterioară:* Administrare filme și programări (operațiuni CRUD asupra catalogului).

5.2. Diagrama de Activitate (Activity Diagram)

Diagrama de activitate modelează fluxul de control dinamic al unui proces specific, de la punctul de start până la finalizare. A fost modelat cu precizie **Fluxul rezervării unui bilet**, care reprezintă procesul critic (*Core Business Flow*) al aplicației.

Pașii fluxului operațional:

1. **Start:** Utilizatorul accesează meniul de „Achiziție Bilete”.

2. **Decizie (Validare Timp):** Sistemul verifică dacă filmul a început deja.
 - *Ramura A:* Dacă da, fluxul se ramifică spre afișarea unui avertisment visual pe ecran.
 - *Ramura B:* Dacă nu (sau dacă avertismentul este acceptat de client), fluxul continuă în mod normal.

3. **Acțiune:** Sistemul randează harta grafică a sălii de cinema (matricea scaunelor).

4. **Acțiune:** Utilizatorul face click pentru a selecta locurile dorite. Sistemul validează automat disponibilitatea acestora în timp real.

5. **Acțiune:** Locurile sunt adăugate în coșul temporar de cumpărături.

6. **Decizie:** Utilizatorul alege una dintre opțiuni:

- *Anulare*: Eliberează imediat locurile blocate din memorie (Rollback).
 - *Finalizare*: Confirmă achiziția biletelor.
7. **Finalizare**: Sistemul calculează prețul final prin motorul de prețuri, scrie tranzacția în fișierele de persistență (.txt) și încheie procesul.

5.3. Diagrama de Secvență (Sequence Diagram)

Diagrama de secvență ilustrează interacțiunea cronologică dintre obiectele sistemului pentru a realiza un anumit caz de utilizare (în acest scenariu, **Achiziția unui bilet**). Această diagramă subliniază schimbul de mesaje între straturile arhitecturale ale aplicației (*Presentation, Controller, Domain, Database*).

Secvența cronologică a mesajelor:

1. **Utilizatorul** trimite un mesaj de input (eveniment click de mouse) către **Aplicație (Stratul de Prezentare)** pentru a selecta un scaun.
2. Stratul de prezentare transmite evenimentul către **Controller (main.cpp)**, care interoghează obiectul din clasa Proiectie (Stratul de Domeniu) pentru a verifica starea locului.
3. Instanța clasei Proiectie evaluează matricea din memoria RAM și returnează un mesaj de confirmare boolean (True / False).
4. La declanșarea comenzii de finalizare, **Controller-ul** transmite datele către clasa BazaDate.
5. Modulul BazaDate deschide un flux de scriere (std::ofstream), serializează obiectele în format text delimitat și transmite datele către **Fișierul Text (Sistemul de Operare)**.
6. Sistemul confirmă succesul persistării, iar Controller-ul afișează confirmarea tranzacției înapoi pe ecranul utilizatorului.

5.4. Diagrama Claselor (Class Diagram)

Diagrama claselor reprezintă structura statică a sistemului, ilustrând entitățile (clasele), atributele lor interne, metodele expuse și relațiile logice (asocieri, compoziții, agregări) dintre ele. Pentru a menține codul C++ modular, sistemul a fost proiectat pe baza următoarelor concepte:

- **User / Admin**: Clase sau structuri responsabile cu gestionarea credențialelor de acces, stocarea parametrilor de sesiune și a stării curente de vizualizare în interfață.
- **Movie (Film)**: Încapsulează detaliile tehnice și comerciale ale peliculei.
 - *Atribute principale*: titlu, durata, gen, pretBaza, estePremiera.

- **CinemaHall (Sala):** Definește topologia și capacitatea încăperii.
 - *Atribute principale:* idSala, numeSala, numarRanduri, numarColoane, tipSala (Standard, IMAX, VIP).
- **Reservation & Seat (Proiectie):** Funcționează ca o clasă de asociere complexă. Leagă un obiect Film de o anumită Sala la o instanță specifică de timp (oraProiectie). Conține logica matriceală booleană pentru starea scaunelor și expune metodele rezervaLoc() și elibereazaLoc().
- **Ticket & Payment (ProdusCos & Motorul de Gestiune):** Structuri care memorează temporar datele tranzacției înainte de checkout. Prețul final nu este unul static, ci este calculat polimorfic la runtime prin metode din Gestiune.h pe baza regulilor din promotii.txt.

CAPITOLUL VI: Implementarea Sistemului

Acest capitol detaliază aspectele practice ale scrierii codului și modul în care au fost implementate funcționalitățile principale ale sistemului. Implementarea a fost realizată integral în limbajul C++, utilizând standardele moderne de gestiune a memoriei și biblioteca ncurses pentru randarea interfeței grafice.

6.1. Modulul de Rezervare

Modulul de rezervare reprezintă componenta de bază (*Core Module*) a interacțiunii cu clienții. Acesta a fost implementat ca un flux de lucru secvențial, ghidând utilizatorul de la alegerea proiecției până la confirmarea finală a tranzacției.

6.1.1. Alegerea filmului

Implementarea acestui sub-modul a necesitat o sincronizare fină între datele statice (cataloagele de filme) și datele dinamice (programul zilei):

- **Extragerea datelor:** Aplicația citește la inițializare fișierele filme.txt și program.txt. Folosind containere din *Standard Template Library* (STL), precum `std::vector<Proiectie>`, sistemul assemblează în memorie programul valid pentru ziua curentă.
- **Afișarea în interfață:** Funcția de randare (`Ecrane::afiseazaBilete`) mapează vectorul de

proiecții pe ecran sub forma unor butoane interactive. Fiecare buton este generat dinamic și include un motor intern de prețuri care verifică dacă în momentul afișării este activă o ofertă specială.

- **Validarea temporală:** Un algoritm evaluează timpul sistemului (<ctime>) comparativ cu ora de începere a peliculei. Filmele expirate sunt ascunse automat, iar pentru cele care au început deja (cu o întârziere tolerată de maximum 30 de minute), implementarea declanșează un *Event Hook* care schimbă culoarea butonului în roșu și forțează afișarea unui meniu de confirmare a întârzierii.

6.1.2. Selectarea locurilor

Selectarea locurilor transpune o matrice logică din memorie într-o grilă vizuală interactivă pe ecran:

- **Logica matricială:** Clasa Proiectie deține o matrice bidimensională (vector de vectori de tip bool) care memorează starea fiecărui scaun. La deschiderea ferestrei sălii, sistemul corelează această matrice cu fișierul rezervari_locuri.txt pentru a marca locurile deja vândute.
- **Interacțiunea hardware:** Utilizând masca de evenimente mousemask din ncurses, sistemul interceptează click-urile. O funcție matematică mapează coordonatele absolute ale terminalului (x, y) la indecșii logici ai matricii (rand, coloana).
- **Gestionarea stărilor:** La fiecare click valid, starea scaunului se comută (*Toggle*). Vizual, implementarea aplică funcții de redare grafică (attron, attroff) pentru a schimba textul din verde ([01] - liber) în alb intens ([SS] - selectat în sesiunea curentă). Scaunele ocupate ([XX]) ignoră complet interceptția mouse-ului.

6.1.3. Confirmarea rezervării

Acest pas reprezintă tranziția datelor din memoria volatilă (Sesiune) în memoria non-volatilă (Fișiere text):

- **Sistemul de Coș (Cart):** Locurile selectate sunt împachetate în obiecte temporare și adăugate în structura vector<ProdusCos> cos. Utilizatorul poate naviga liber prin aplicație (de exemplu, să meargă la Snack Bar), coșul păstrându-și starea curentă.
- **Finalizarea tranzacției (Commit):** La apăsarea butonului de finalizare, sistemul declanșează o operațiune de scriere. Implementarea deschide fișierele de persistență

(vanzari_bilete.txt și rezervari_locuri.txt) folosind fluxuri std::ofstream cu parametrul ios::app (*Append*). Fiecare loc rezervat și bilet emis este transformat într-un șir de caractere delimitat de pipe (|) și scris pe o linie nouă. După scrierea cu succes, memoria coșului este eliberată (cos.clear()), iar interfața redirecționează utilizatorul la meniul principal.

6.2. Modulul de Administrare

Componenta de *Back-Office* a fost implementată pentru a asigura trasabilitatea financiară și operațională a cinematografului. Aceasta permite managementului să auditeze operațiunile de vânzare fără a părăsi interfața TUI.

6.2.1. Capabilitatea de a vedea un istoric al vânzărilor

Pentru a oferi informații relevante și ușor de parcurs, istoricul vânzărilor nu realizează o simplă afișare text brută a fișierelor de baze de date, ci implementează un algoritm dedicat de agregare și procesare a datelor:

- **Mecanismul de Securitate:** Modulul este protejat în mod strict de starea ADMIN_LOGIN. Implementarea utilizează o citire secvențială a caracterelor de la tastatură, înlocuindu-le vizual cu caractere de mascare (*), protejând astfel parola implicită (cinema2026) împotriva vizualizării accidentale (*shoulder surfing*).
- **Parsarea și Colectarea Datelor:** Sistemul citește liniile din fișierele vanzari_bilete.txt și vanzari_bar.txt. Folosind fluxul std::stringstream, datele brute sunt fragmentate în jetoane (*token-uri*).
- **Agregarea și Sortarea Inteligentă:** Pentru a preveni aglomerarea ecranului cu înregistrări repetitive (ex. zeci de porții de popcorn vândute individual la aceeași oră), algoritmul utilizează structura std::map<string, int>. Pentru a forța o sortare cronologică (deoarece o structură map în C++ ordonează implicit alfabetic), cheia fiecărei intrări este construită prin concatenarea inversată: string cheieSortare = YYYYMMDD + "|" + HH:MM + "|" + Nume_Entitate;
- **Comasarea datelor:** Dacă două produse identice sunt vândute în cadrul aceluiași minut, algoritmul identifică cheia existentă în map, incrementează contorul cantității (ex. Buc: 2) și adună corespunzător valoarea financiară.
- **Randarea Rapoartelor:** Odată ce procesarea structurilor s-a încheiat, sistemul randează tabele aliniate folosind funcția snprintf pentru formatarea monospațiată a coloanelor. Administratorul poate utiliza mecanismul de scroll al mouse-ului pentru a naviga prin tot istoricul financiar, beneficiind de un control de paginare optimizat, identic

cu cel utilizat în interfața destinată clienților.

CAPITOLUL VII: Concluzii și Direcții Viitoare de Dezvoltare

Interfața om-mașină (HCI - Human-Computer Interaction) reprezintă punctul de contact dintre sistemul software și utilizatorul final. Deși aplicația este dezvoltată în C++ și rulează în terminal, interfața nu este una de tip linie de comandă (CLI), ci o interfață grafică text (TUI - Text-based User Interface) avansată, implementată cu ajutorul bibliotecii ncurses. Aceasta suportă navigare asincronă prin mouse, coduri de culori și elemente vizuale de tip "fereastră", asigurând o ergonomie ridicată.

În subcapitolele următoare sunt prezentate și descrise ecranele principale ale aplicației (stările mașinii de stări finite).

7.1. Pagina principală (Meniul de Navigare)

Pagina principală acționează ca un nod central de rutare (Hub). Designul este minimalist, pentru a nu suprasolicita cognitiv utilizatorul. Pe ecran sunt randate centralizat opțiunile principale de navigare, sub forma unor butoane interactive supradimensionate. În colțul din dreapta-sus este poziționat permanent indicatorul pentru „Coșul de cumpărături”, iar în stânga-sus butonul securizat pentru accesul personalului (Administrator).

Designul utilizează perechi de culori neutre (ex. text alb pe fundal albastru închis) pentru a oferi un aspect profesional.



Figura 7.1 - Ecranul principal de navigare al sistemului, evidențiind butoanele interactive și structura meniului.

7.2. Pagina filmelor (Programul Cinematografic)

Acest ecran prezintă utilizatorului lista dinamică a proiecțiilor disponibile pentru ziua curentă.

- **Aspect vizual:** Filmele sunt afișate sub formă de listă verticală (List View). Din cauza limitărilor de spațiu ale terminalului, lista este paginată, utilizatorul putând folosi roțița mouse-ului pentru a face scroll.
- **Codificarea culorilor:** Interfața folosește culori semantice pentru a transmite informații rapid. Eticheta [PREMIERA] este colorată în magenta, iar [VIP] în galben.
- **Feedback interactiv:** La trecerea cursorului sau la click, butonul selectat oferă feedback vizual (prin funcția `attron(A_REVERSE)` sau modificarea perechii de culori).

(NOTĂ: Aici se va insera Captura de Ecran 2 - Pagina Filmelor) **Figura 7.2** - Afișarea catalogului de filme, incluzând etichetele de categorie și prețurile calculate dinamic.

7.3. Detalii film și Validarea temporală

La selectarea unui film din listă, interfața grafică afișează un sumar al peliculei (Titlu, Gen, Durată estimată și Sala alocată). O caracteristică importantă de UI/UX implementată în acest ecran este mecanismul de avertizare temporală.

Dacă sistemul detectează că filmul a început deja de câteva minute, interfața deschide o „fereastră modală” de tip Pop-up. Elementele grafice devin roșii (semnalizând o alertă), iar utilizatorul este obligat să citească avertismentul și să confirme explicit intenția de a continua rezervarea.

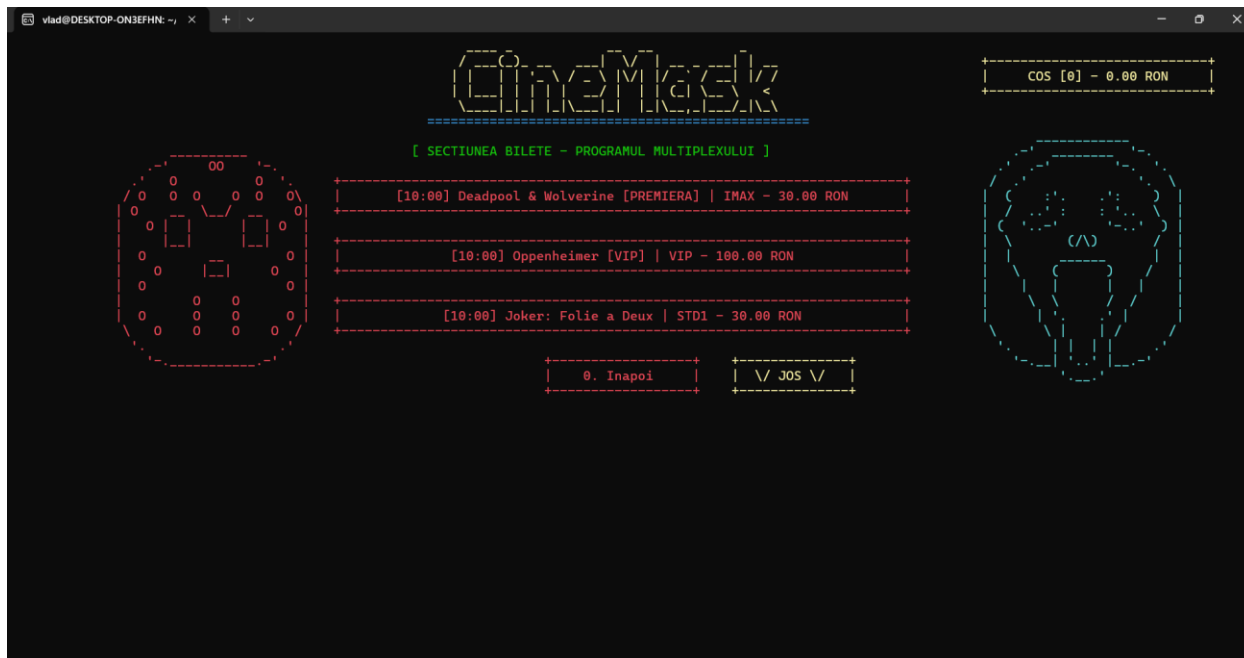


Figura 7.3 - Ecranul de detalii ale proiecției, evidențiind fereastra de avertizare pentru filmele care au început deja.

7.4. Selectarea locurilor (Seat Mapping)

Acesta este cel mai complex ecran din punct de vedere al randării grafice. Interfața translatează o matrice din memorie într-o grilă interactivă reprezentând fizic sala de cinema.

- **Reprezentarea scaunelor:** Terminalul desenează o grilă spațiată. Locurile libere sunt afișate sub forma [01], colorate cu verde pentru a indica disponibilitatea.
- **Starea de ocupare:** Locurile citite din baza de date ca fiind deja vândute sunt transformate în [XX] și colorate în roșu, iar sistemul blochează complet interacțiunea mouse-ului cu acele coordonate grafice.
- **Selecția curentă:** Atunci când utilizatorul face click pe un loc liber, acesta se preschimbă vizual în [SS] (text alb strălucitor / evidențiat), confirmând adăugarea temporară a locului în memorie.

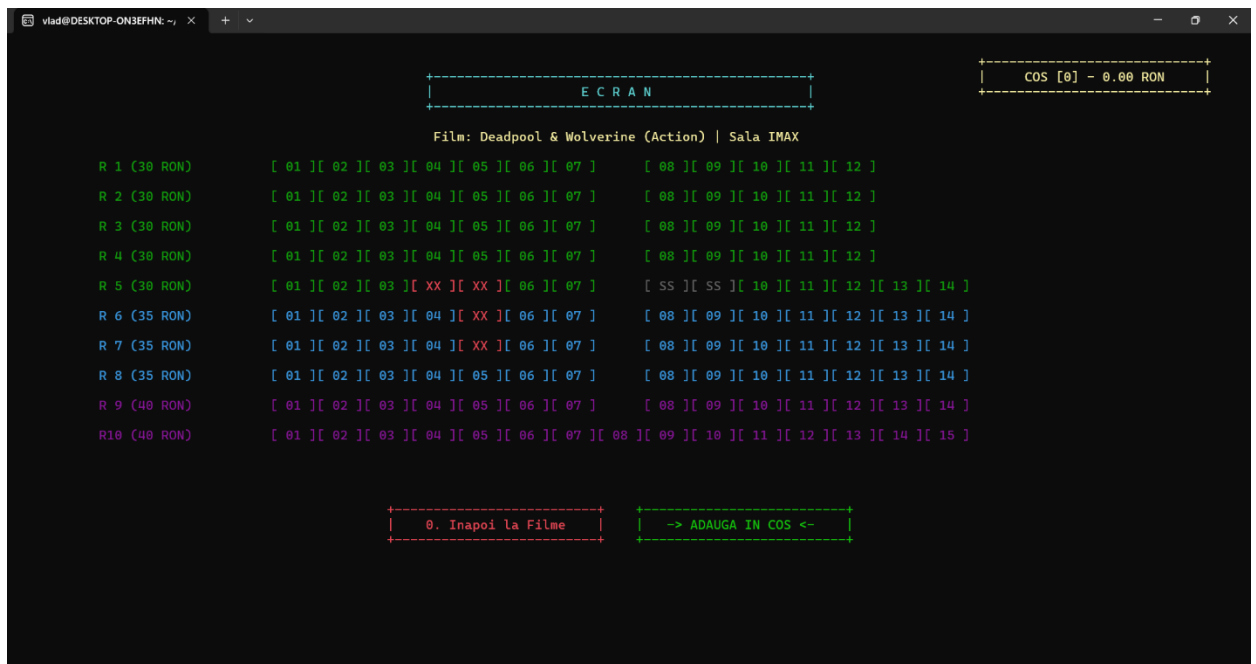


Figura 7.4 - Matricea interactivă a sălii de cinema, demonstrând diferențierea vizuală între locurile libere, ocupate și selectate.

7.5. Coșul de cumpărături și Confirmare plată (Checkout)

Ecranul de coș (Cart) acționează ca un sumar financiar înainte de finalizarea tranzacției (Commit).

- **Structura:** Partea stângă a ecranului listează toate produsele selectate: biletele (cu specificarea exactă a filmului, sălii, rândului și locului) și produsele de bar.
- **Prețuri dinamice:** În partea dreaptă, sistemul afișează totalul de plată. Dacă motorul de promoții a detectat o regulă valabilă (ex. Vineri preț fix, sau reducere procentuală), interfața grafică afișează suma originală tăiată și noul preț, alături de o notificare clară (ex. [OFERTĂ APLICATĂ]).
- **Finalizare:** Un buton mare de „Finalizează Comanda” stă la baza ecranului, a cărui apăsare declanșează scrierea datelor pe disc și confirmarea succesului.



Figura 7.5 - Sumarul coșului de cumpărături, afișând calculul dinamic al prețului și butonul de finalizare a comenzii.

7.6. Panou administrator (Back-Office / Audit)

Interfața de administrare schimbă paradigma vizuală de la "client-friendly" la un aspect de tip dashboard analitic, optimizat pentru densitatea informației.

- **Ecranul de Login:** La accesare, interfața cere o parolă. Securitatea UI este asigurată prin randarea caracterului * la fiecare apăsare de tastă, ascunzând datele introduse.
- **Tabelele de Raportare:** Odată autentificat, managerul are acces la rapoarte. Ecranul folosește caractere ASCII/Unicode pentru a desena tabele structurate (Linii și Coloane clare).
- **Vizualizare aglomerată:** Datele extrase sunt formatate cu precizie (folosind spațieri monospațiate `sprintf`) pentru a alinia coloanele de Film/Produs, Data, Ora, Cantitatea și Încasările. Suportul pentru scroll permite navigarea prin sute de înregistrări fără ca ecranul să se „spargă” sau să piardă alinierea.

FILM	ORA	SALA	DATA	BILETE VANDUTE	INCASARI TOTALE
1. Deadpool & Wolverine	10:00	IMAX	31/05/2026	Bilete: 3	100.00 RON
2. Interstellar	10:00	IMAX	24/05/2026	Bilete: 7	245.00 RON
3. The Batman	10:00	STD1	24/05/2026	Bilete: 6	150.00 RON
4. Avatar: Way of Water	12:42	STD2	24/05/2026	Bilete: 7	175.00 RON
5. Inception	13:19	IMAX	24/05/2026	Bilete: 6	210.00 RON
6. Spider-Man: No Way Home	13:26	STD1	24/05/2026	Bilete: 6	150.00 RON

0. Inapoi V JDS V

Figura 7.6 - Interfața modulului de Back-Office, prezentând tabelul agregat al istoricului de vânzări ordonat cronologic.

BIBLIOGRAFIE

1. **Dickey, Thomas E.**, *NCURSES - New Curses Library Manual and Developer's Guide*, GNU Project, 2024. [Sursă online: <https://invisible-island.net/ncurses/>]
2. **Stroustrup, Bjarne**, *The C++ Programming Language (4th Edition)*, Addison-Wesley Professional, 2013. (Referință fundamentală pentru utilizarea avansată a Standard Template Library - std::vector, std::map și gestiunea memoriei).
3. **Martin, Robert C.**, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017. (Referință teoretică pentru implementarea Arhitecturii pe Straturi / N-Tier, a mașinilor de stări finite și pentru decuplarea codului sursă).
4. **Cplusplus.com & Cppreference.com**, *Standard C++ Library Reference*. Documentație oficială pentru manipularea fluxurilor de fișiere (<fstream>), parsarea șirurilor de caractere (<sstream>) și gestiunea timpului curent (<ctime>).
5. **Google Gemini (Model AI Generativ)**, *Asistență tehnică și consultanță arhitecturală*. Utilizat ca asistent virtual de programare pentru optimizarea algoritmilor de sortare

cronologică (std::map key injection), refactorizarea modulelor C++ și structurarea, respectiv redactarea academică a prezentei documentații tehnice. [Sursă online: <https://gemini.google.com/>]